

Basics of Programming II. Project

Course name: Basics of Programming II.

Course ID: ENVIAUAA01

Full name: Noor Ali Jawad Almukhtar

Neptun code: [REDACTED]

Group ID: EE16D

Name of the lab instructor: [REDACTED]

Project title: SongKeeper

Type of the project: Final Project

Submission date: 24/05/2026

Introduction

The purpose of this project is to create a simple song management system called **SongKeeper** using C++. The program allows the user to add, search, delete, save, and load songs through a menu-driven interface. The project was developed using object-oriented programming concepts learned in the Basics of Programming II course.

- Each song contains information such as:
 - Title
 - Artist
 - Album
 - Genre
 - Year
- The project also uses important C++ concepts, including:
 - Classes and objects
 - Constructors and destructors
 - Dynamic memory allocation
 - File handling
 - Exception handling
- The program was implemented using three source files:
 - SongKeeper.h
 - SongKeeper.cpp
 - Main.cpp

Program Interface

- The program is executed in Visual Studio Code using the terminal window and the TDM-GCC compiler.
- Compiler command: `g++ main.cpp SongKeeper.cpp -o SongKeeper.exe`
- Run command: `.\SongKeeper.exe`
- The program displays a menu that lets the user choose different operations. The program is terminated when the user selects the Exit option.

Program Execution

- When the program starts, it automatically loads previously saved songs from songs.txt if the file exists.
- After starting the program, the following menu is displayed:
 1. Search a song.
 2. Add a song.
 3. Delete a song.
 4. Save to file.
 5. Exit.

➔ 1. Search a song:

- The user can search for songs by:
 - Title
 - Artist
 - Album
 - Genre
 - Year
- If a matching song is found, the program prints all information about the song.
- If no song is found, the program displays an error message.

➔ 2. Add a song:

- The user enters:
 - Title
 - Artist
 - Album
 - Genre
 - Year
- The program creates a new Song object and stores it dynamically inside the collection.
- Duplicate songs are checked using the title and artist.

➔ 3. Delete a song:

- The user enters the title of the song to delete.
- If multiple songs with the same title are found, the program displays all matching songs and asks the user which one should be deleted, or the user can go back.
- Example:

- 1. Hello – Adele
 - 2. Hello – Lionel Richie
 - 0. Back
 - The selected song is removed from memory, and the save file is updated automatically.
- ➔ 4. Save to file:
- The program saves all songs into a text file called: songs.txt
 - Each song is stored in one line, separated by semicolons.
- ➔ 5. Exit:
- Before the program closes, the current collection is automatically saved into the file.

Input and Output

- The program receives input from the keyboard using: cin, getline().
- Example: entering song details and the output will be "Song added."
- The program also performs file handling using: ifstream, ofstream.

Program Structure

- The project consists of three source files.
- ➔ SongKeeper.h
 - This file contains:
 - Class declarations
 - Private data members
 - Function declarations
 - Two classes are used:
 - Song
 - SongKeeper
 - The Song class stores information about one song, while the SongKeeper class manages the song collection.

➔ SongKeeper.cpp

- This file contains all function implementations such as:
 - Adding songs
 - Searching songs
 - Deleting songs
 - Resizing the dynamic array
 - Saving songs
 - Loading songs
- The program uses dynamic memory allocation:
 - new, new[]
 - delete, delete[]
- Songs are stored using: Song** songs;
- Which represents a dynamic array of pointers.
- The program also uses a toLower() function to support case-insensitive searching and deleting.

➔ main.cpp

- This file contains:
 - the main menu
 - user interface
 - input handling
 - exception handling
- The program execution starts from the main() function.

Testing and Verification

- The program was tested using different scenarios, such as:
 - adding songs
 - searching songs
 - deleting songs
 - saving and loading files
 - duplicate checking
 - deleting songs with the same title
 - invalid user input
- The program was also tested after restarting to verify that songs are correctly loaded from songs.txt.
- Exception handling was tested by:
 - attempting to add duplicate songs
 - deleting non-existing songs
 - using invalid year values

Improvements and Extensions

- Possible future improvements include:
 - graphical user interface
 - editing songs
 - playlists
 - sorting songs
 - adding songs duration and lyrics
- The current version uses a simple terminal-based interface, but additional features can be added in the future.

Difficulties Encountered

- Some difficulties encountered during the project were:
 - using pointers correctly
 - implementing case-insensitive searching
 - improving the delete function for songs with the same title
 - configuring VS Code build settings

Conclusion

- This SongKeeper project successfully implements a simple song management system using C++. The project applies key programming concepts, including object-oriented programming, dynamic memory management, file handling, and exception handling.
- The final program allows users to manage songs through a simple menu system while saving song data permanently in a text file.

References

- Basics of Programming II lecture and laboratory materials, Budapest University of Technology and Economics.